

RAPPORT DÉTAILLÉ

Guezzane, Ahmed Fateh
BOIS-DE-BOULOGNE

TABLE DES MATIÈRES

Introduction.....	3
DBConnector	4
Description.....	4
Méthodes et descriptions.....	4
Les DAO	5
AccountDAO_InMemory	5
Description.....	5
Méthodes et descriptions.....	5
RecipeDAO_InDB.....	7
Description.....	7
Méthodes et descriptions.....	7
Points clés	8
Les Services	9
AccountService	9
Description.....	9
Détail des méthodes de la classe	9
Résumé de la classe	10
RecipeService	11
Description.....	11
Détail des méthodes de la classe	11
Résumé de la classe	12
Les Servlets.....	14
AccountServlet.....	14
Description.....	14
Détail des méthodes de la classe	14
Résumé de la classe	14
GetLocaleRequest.....	16

Description.....	16
Détail de la classe.....	16
Résumé de la classe	16
LoadDashboardPage	17
Description.....	17
Détail des méthodes de la classe	17
Résumé de la classe	17
LoginServlet.....	18
Description.....	18
Détail des méthodes de la classe	18
Résumé de la classe	18
RecipeServlet	20
Description.....	20
Détail des méthodes de la classe	20
Résumé de la classe	20
Base de données.....	22
Vue d'ensemble de la structure.....	22
Model	23
Account.....	23
Ingredient	23
Rating.....	24
Recipe	24
Restriction.....	25
Scripts	26
Scripts de adminPanel.jsp	26
Scripts de recipes-search.jsp	27
Internationalisation	28
Conclusion	29

Introduction

Notre application web est un système complet de gestion de recettes et d'utilisateurs. Elle permet aux utilisateurs de consulter des recettes, de les filtrer par catégories, restrictions et ingrédients, et de rechercher des recettes par ingrédient spécifique. Les utilisateurs enregistrés peuvent ajouter des recettes à leur liste d'épicerie. L'application calcule ensuite le total des ingrédients et génère une liste complète des éléments nécessaires, que l'utilisateur peut imprimer au format .pdf. Seuls les utilisateurs enregistrés ont accès à cette fonctionnalité, tandis que les utilisateurs non enregistrés peuvent consulter les recettes et accéder aux instructions interactives de chaque recette. L'application inclut également un système de gestion d'administrateurs permettant de gérer les comptes d'utilisateurs, les recettes et les catégories de filtres pour affiner les recherches de recettes.

L'architecture de l'application suit le modèle MVC (Modèle-Vue-Contrôleur). Le modèle (Model) gère les données et la logique de l'application, comme les recettes et les utilisateurs. Les services (Service) agissent comme intermédiaires entre le modèle et le contrôleur, traitant des opérations complexes avant de transmettre les informations au contrôleur. Le contrôleur (Controller) gère les requêtes des utilisateurs, interagit avec les services et transmet les données aux vues (JSP). Les DAOs (DAL) assurent la communication avec la base de données, facilitant l'accès aux données nécessaires. Les fichiers JSP associés au tableau de bord sont sécurisés dans le répertoire WEB-INF, tandis que les autres pages JSP sont accessibles dans le répertoire "web pages". Les fichiers CSS et les images sont centralisés dans le dossier "Assets", permettant une gestion claire et un accès rapide.

DBConnector

Description

La classe DBConnector est utilisée pour établir une connexion avec une base de données MariaDB. Elle suit le patron de conception Singleton, ce qui garantit qu'une seule instance de la connexion à la base de données existe au sein de l'application. Cette classe est responsable de la gestion de la connexion et de la fourniture de cette connexion à d'autres parties de l'application lorsqu'elle est nécessaire.

Méthodes et descriptions

Méthode	Description	Type de retour	Arguments	Processus logique
GetInstance()	Retourne l'instance unique de la classe dbconnector (singleton). Si l'instance n'existe pas, elle est créée.	Dbconnector	Aucun	Vérifie si une instance existe, sinon en crée une nouvelle, puis la retourne.
Dbconnector()	Constructeur privé initialisant la connexion à la base de données. Charge le driver jdbc et établit la connexion.	Aucun	Aucun	Charge le driver jdbc, tente de se connecter à la base de données avec les paramètres de connexion définis.
Getconnection()	Retourne la connexion active à la base de données.	Connection	Aucun	Retourne l'objet connection établi avec la base de données. Si la connexion a échoué dans le constructeur, elle serait null.

Les DAO

AccountDAO_InMemory

Description

La classe AccountDAO_InMemory implémente l'interface IAccountDAO et fournit des méthodes pour gérer les comptes d'utilisateurs, leurs mots de passe, et leurs listes de courses dans un stockage en mémoire (via RegistreAccount). Elle prend en charge des opérations de lecture, création, suppression, et mise à jour des comptes d'utilisateurs, ainsi que la gestion des listes de courses des utilisateurs.

Méthodes et descriptions

Method Name	Description	Return Type	Arguments	Logical Process
fetchAccountByUsername	Retrieves a user account based on the username.	Account	String username	Filters the list of accounts to find a match with the provided username. Returns the account if found, otherwise null.
fetchAccountList	Returns the list of all user accounts registered in the registry.	List<Account>	None	Accesses the list of accounts stored in RegistreAccount and returns it.
fetchGroceryList	Retrieves a user's grocery list based on their user ID.	List<Recipe>	String userId	Filters accounts to find the one matching the user ID and returns the associated grocery list. Returns null if no account is found.
submitAccount	Submits a new account to the list of accounts after checking for duplicate usernames or emails.	int	Account account	If no account with the same username or email exists, the account is added to the list and returns 1. If a duplicate exists, returns 0.
deleteAccount	Deletes a user account based on the user ID.	int	String userId	Searches for the account with the given user ID and deletes it if found. Returns 1 if successful, -1 if the user is not found.
changePassword	Allows a user to change their password by providing the old and new passwords.	int	String userId, String oldPassword, String newPassword	Verifies if the old password matches the stored one. Returns -2 if not, 0 if old and new passwords are the same, or 1 if the password is updated successfully.
approveAccount	Approves a user account by changing its status (e.g., marking it as activated).	int	String userId	Searches for the account with the given user ID. If already approved (status 1), returns 0. Otherwise, changes the status to 1 and returns 1.

addToGroceryList	Adds a recipe to a user's grocery list.	int	String userId, String recipeId	Finds the user account and adds an empty recipe to their grocery list. Returns 1 if successful, 0 if the user is not found.
removeFromGroceryList	Removes a recipe from a user's grocery list based on the recipe ID.	int	String userId, String recipeId	Finds the user account and attempts to remove the recipe matching the recipeId. Returns 1 if successful, 0 if the recipe is not found.
clearGroceryList	Clears a user's grocery list.	int	String userId	Finds the user account and clears their grocery list. Returns 1 if successful, 0 if the user is not found.
verifyPassword	(Not yet implemented) Verifies if a password is valid for a given username.	void	String username, String password	Not implemented.

RecipeDAO_InDB

Description

La classe RecipeDAO_InDB implémente l'interface IRecipeDAO et est responsable de la gestion des opérations CRUD (Create, Read, Update, Delete) pour les recettes dans la base de données. Elle utilise une connexion à la base de données, gérée par DBConnector, pour effectuer des requêtes SQL afin de récupérer, insérer, mettre à jour ou supprimer des données relatives aux recettes et à leurs composants associés, tels que les ingrédients, les instructions, les restrictions et les évaluations.

Méthodes et descriptions

Méthode	Description	Retour	Arguments	Processus
fetchRecipeList	Récupère la liste complète des recettes de la base de données.	List<Recipe>	Aucun	Exécute une requête SQL pour obtenir les informations basiques de toutes les recettes.
fetchRecipeByTitle	Récupère une recette spécifique par son titre.	Recipe	String title	Exécute une requête SQL pour récupérer les détails d'une recette et charger ses composants associés.
fetchIngredientsById	Récupère les ingrédients d'une recette par son ID.	List<Ingredient>	String recipeId	Exécute une requête SQL pour obtenir les ingrédients associés à la recette.
fetchInstructionsByRecipeId	Récupère les instructions d'une recette.	List<String>	String recipeId	Exécute une requête SQL pour obtenir les instructions et les divise en une liste.
fetchRestrictionsByRecipeId	Récupère les restrictions alimentaires d'une recette.	List<Restriction>	String recipeId	Exécute une requête SQL pour obtenir les restrictions par ID de recette.
fetchRatingsByRecipeId	Récupère les évaluations d'une recette.	List<Rating>	String recipeId	Exécute une requête SQL pour récupérer les évaluations par ID de recette.
fetchRecipesByRestriction	Récupère les recettes associées à une restriction spécifique.	List<Recipe>	String restrictionType	Exécute une requête SQL pour obtenir les recettes liées à une restriction donnée.
fetchRecipesByCategory	Récupère les recettes d'une catégorie spécifique.	List<Recipe>	String category	Exécute une requête SQL pour obtenir les recettes de la catégorie spécifiée.
fetchRecipesWithAvgRatingOver	Récupère les recettes avec une note moyenne supérieure à un seuil.	List<Recipe>	double ratingThreshold	Exécute une requête SQL pour obtenir les recettes ayant une note moyenne supérieure au seuil.

fetchRecipeById	Récupère une recette par son ID.	Recipe	String id	Exécute une requête SQL pour obtenir les détails d'une recette et charger ses composants associés.
fetchRecipesByStatus	Récupère les recettes ayant un statut spécifique.	List<Recipe>	int status	Exécute une requête SQL pour obtenir les recettes selon leur statut.
fetchRecipesByIngredients	Récupère les recettes contenant certains ingrédients.	List<Recipe>	List<String> ingredientNames	Génère une clause WHERE dynamique pour filtrer les recettes par ingrédients et exécute la requête SQL.
submitRecipe	Ajoute une nouvelle recette dans la base de données.	int	Recipe recipe	Exécute une requête SQL pour insérer les détails de la recette.
addRestriction	Ajoute des restrictions alimentaires à une recette.	int	String recipeId, List<Restriction> restrictionList	Exécute une requête SQL pour insérer des restrictions associées à la recette.
addIngredients	Ajoute des ingrédients à une recette.	int	String recipeId, List<Ingredient> ingredientList	Exécute une requête SQL pour insérer les ingrédients associés à la recette.
addRating	Ajoute une évaluation d'un utilisateur à une recette.	int	String userId, String recipeId, int rating	Exécute une requête SQL pour insérer une évaluation pour la recette.
deleteRecipeByTitle	Supprime une recette par son titre.	int	String title	Exécute une requête SQL pour supprimer la recette correspondant au titre spécifié.
deleteRecipeById	Supprime une recette par son ID.	int	String id	Exécute une requête SQL pour supprimer la recette identifiée par son ID.
modifyRecipe	Modifie une recette existante.	int	Recipe recipe	Exécute une requête SQL pour mettre à jour les détails d'une recette existante.

Points clés

- La classe RecipeDAO_InDB encapsule toutes les opérations liées à l'accès aux données de recettes dans une base de données, notamment en ce qui concerne l'ajout, la récupération, la mise à jour et la suppression des recettes et de leurs composants.
- Elle gère également des objets complexes comme les ingrédients, les instructions, les restrictions et les évaluations associées à chaque recette.

Les Services

AccountService

Description

La classe AccountService est responsable de la gestion des opérations liées aux comptes utilisateurs dans une application, en particulier pour l'interaction avec l'interface IAccountDAO (Data Access Object) pour effectuer des opérations sur les données des utilisateurs. Elle agit comme un intermédiaire entre le contrôleur (ou l'interface utilisateur) et le DAO, en gérant la logique métier et en assurant que les données circulent correctement entre les différentes couches de l'application.

Les principales fonctionnalités de la classe incluent la création de comptes, la gestion des mots de passe, l'approbation ou la suppression des comptes, ainsi que la gestion des listes de courses des utilisateurs. Elle fournit également des méthodes pour gérer et agréger les ingrédients des recettes.

Détail des méthodes de la classe

Méthode	Description	Retour	Arguments	Processus
fetchAccountByUsername	Récupère un compte par son nom d'utilisateur.	Account	String username	Renvoie un objet Account si trouvé, sinon null.
fetchAccountList	Récupère la liste de tous les comptes.	List<Account>	Aucun	Renvoie la liste complète des comptes depuis la source de données.
fetchAdminAccountList	Récupère uniquement les comptes administrateurs.	List<Account>	Aucun	Filtre la liste des comptes pour ne contenir que les administrateurs avec des champs essentiels.
fetchInactiveAccountList	Récupère les comptes inactifs (status = 0).	List<Account>	Aucun	Renvoie les comptes inactifs sous une forme simplifiée.
fetchGroceryList	Récupère la liste de courses d'un utilisateur.	List<Recipe>	String userId	Renvoie une liste d'objets Recipe appartenant à l'utilisateur spécifié.
submitAccount	Crée et soumet un nouveau compte utilisateur.	int	String username, String firstName, String lastName, String email, String password, String securityQuestion, String securityAnswer, int status	Vérifie si le nom d'utilisateur existe déjà et renvoie -1 si c'est le cas. Soumet le compte si les données sont valides.

addToGroceryList	Ajoute une recette à la liste de courses d'un utilisateur.	int	String userId, String recipeId	Ajoute une recette spécifiée à la liste de courses.
changePassword	Modifie le mot de passe d'un utilisateur.	int	String userId, String oldPassword, String newPassword	Vérifie si l'ancien mot de passe est correct avant de le modifier.
approveAccount	Approuve un compte utilisateur.	int	String userId	Change le statut du compte pour le rendre actif.
deleteAccount	Supprime un compte utilisateur.	int	String userId	Supprime un compte en fonction de l'ID utilisateur.
removeFromGroceryList	Supprime une recette de la liste de courses.	int	String userId, String recipeId	Retire une recette spécifiée de la liste de courses.
clearGroceryList	Vide la liste de courses d'un utilisateur.	int	String userId	Supprime tous les éléments de la liste de courses de l'utilisateur.
verifyPassword	Vérifie la validité d'un mot de passe pour un utilisateur.	boolean	String username, String password	Vérifie si le mot de passe correspond à celui enregistré.
generateIngredientList	Génère une liste consolidée d'ingrédients à partir de plusieurs recettes.	List<Ingredient>	List<Recipe> recipeList	Combine les ingrédients identiques en additionnant leurs quantités.

Résumé de la classe

- But : Cette couche de service est responsable de la gestion des comptes utilisateurs et de leurs actions associées, telles que l'enregistrement, la gestion des mots de passe, l'approbation et la gestion des listes (listes de courses et listes d'ingrédients).
- Dépendances : Elle dépend de IAccountDAO, une interface Data Access Object, qui gère les opérations sur la base de données pour les comptes, les recettes et les ingrédients. La classe AccountService assure que la logique métier est correctement traitée et que les données sont transmises correctement entre la couche de service et la couche DAO.
- Gestion des listes : Le service gère également des fonctionnalités spécifiques pour les utilisateurs liées à leurs listes de courses (ajouter/retirer des recettes) et leurs listes d'ingrédients (agrégation des ingrédients entre plusieurs recettes).

Cette structure permet de séparer les préoccupations de manière efficace, permettant à la classe AccountService de se concentrer sur la logique métier, tandis que la couche DAO se charge des interactions avec la base de données.

RecipeService

Description

La classe RecipeService est responsable de la gestion des opérations liées aux recettes dans une application, en interagissant avec l'interface IRecipeDAO pour effectuer des actions sur les données des recettes. Elle sert de couche intermédiaire entre le contrôleur ou l'interface utilisateur et la couche d'accès aux données (DAO), en traitant la logique métier avant d'effectuer les actions sur la base de données.

Les principales fonctionnalités de la classe incluent la gestion des recettes (création, modification, suppression), l'ajout d'ingrédients et de restrictions aux recettes, ainsi que l'ajout de notes (ratings) et le calcul de la note moyenne des recettes. Elle permet également de récupérer des recettes en fonction de divers critères comme le statut, les ingrédients, les catégories, ou les évaluations.

Détail des méthodes de la classe

Méthode	Description	Retour	Arguments	Processus
fetchRecipeByTitle	Récupère une recette par son titre.	Recipe	String title	Renvoie l'objet Recipe si trouvé, sinon null.
fetchRecipeById	Récupère une recette par son identifiant unique.	Recipe	String id	Renvoie l'objet Recipe si trouvé, sinon null.
fetchRecipesByStatus	Récupère les recettes en fonction de leur statut.	List<Recipe>	int status	Filtre les recettes selon leur statut (actif, inactif, etc.).
fetchRecipeList	Récupère la liste complète des recettes.	List<Recipe>	Aucun	Renvoie toutes les recettes disponibles.
submitRecipe	Crée et soumet une nouvelle recette.	int	String title, String description, String category, String difficulty, double prepTime, double servings, int status	Vérifie si le titre existe déjà. Si oui, retourne -1. Soumet la recette si valide.
deleteRecipeByTitle	Supprime une recette par son titre.	int	String title	Supprime la recette correspondante.
deleteRecipeById	Supprime une recette par son identifiant.	int	String id	Supprime la recette correspondante.
modifyRecipe	Modifie une recette existante.	int	Recipe recipe	Met à jour les informations de la

				recette dans la base de données.
addRestrictionsToRecipe	Ajoute des restrictions à une recette.	int	String recipeId, List<String> restrictionTypes	Crée et associe des restrictions alimentaires à la recette.
addIngredientsToRecipe	Ajoute des ingrédients à une recette.	int	String recipeId, List<Ingredient> ingredients	Associe des ingrédients à la recette spécifiée.
addRating	Ajoute une évaluation (note) à une recette.	int	String recipeId, String userId, int rating	Ajoute une note. Si la recette n'existe pas, retourne -1.
fetchAverageRating	Récupère la note moyenne d'une recette.	double	String recipeId	Calcule la note moyenne. Si la recette est introuvable, retourne 0.0.
fetchRecipesByIngredients	Récupère les recettes contenant des ingrédients spécifiques.	List<Recipe>	List<String> ingredientsList	Filtre les recettes contenant tous les ingrédients donnés.
fetchRecipesByRestriction	Récupère les recettes selon une restriction alimentaire.	List<Recipe>	String restriction	Filtre les recettes en fonction de la restriction (végétarien, sans gluten, etc.).
fetchRecipesByAverageRating	Récupère les recettes avec une note moyenne minimum.	List<Recipe>	String rating	Retourne les recettes avec une note supérieure ou égale à la valeur spécifiée.
fetchRecipesByCategory	Récupère les recettes par catégorie.	List<Recipe>	String category	Filtre les recettes selon leur catégorie (entrée, plat, dessert, etc.).

Résumé de la classe

- But : Cette classe assure la gestion des recettes dans le cadre d'une application de gestion de recettes, en permettant des opérations telles que l'ajout, la suppression, la modification, et la consultation des recettes. Elle permet également de gérer les évaluations, les restrictions et les ingrédients des recettes.
- Dépendances : La classe dépend de IRecipeDAO, qui est une interface d'accès aux données permettant d'interagir avec la base de données. La classe RecipeService manipule les objets Recipe, Ingredient, Rating, et Restriction tout en déléguant les opérations de base de données au DAO.

- **Fonctionnalités** : Elle fournit un ensemble d'outils permettant à l'utilisateur ou à l'administrateur de rechercher, gérer et modifier les recettes, d'ajouter des évaluations, et de filtrer les recettes par différents critères tels que les ingrédients, les restrictions ou les catégories.

En résumé, cette classe sert de couche métier pour les opérations liées aux recettes, en orchestrant les interactions entre la logique de l'application et les données stockées dans le backend (base de données).

Les Servlets

AccountServlet

Description

La classe AccountServlet est un contrôleur de servlet dans une application Java EE, chargée de gérer les actions liées au compte utilisateur. Elle interagit avec le service AccountService pour effectuer des opérations sur les comptes, comme l'enregistrement, la connexion, la gestion des listes de courses, et la génération de rapports PDF. Elle traite les actions demandées par l'utilisateur, effectuées via les requêtes HTTP, et redirige l'utilisateur vers les pages appropriées.

Les actions possibles sont, entre autres, la déconnexion, l'affichage du tableau de bord d'administration, l'enregistrement d'un nouveau compte, l'ajout ou la suppression d'ingrédients dans une liste de courses, et l'impression de la liste des ingrédients en PDF.

Détail des méthodes de la classe

Action	Description	Processus
logout	Déconnexion de l'utilisateur.	Met à jour la session en définissant logged à false, account à null et isAdmin à null. Redirige vers index.jsp.
dashboard	Accès au tableau de bord administrateur.	Redirige l'utilisateur administrateur vers la page adminPanel.jsp.
register	Inscription d'un nouvel utilisateur.	Récupère les données de l'utilisateur. Si le mot de passe et la confirmation ne correspondent pas, redirige vers la page d'inscription avec un message d'erreur. En cas de succès, redirige vers la page de connexion.
addToGrocery	Ajoute une recette à la liste de courses.	Extrait l'utilisateur et la recette de la session, ajoute la recette à la liste de courses via AccountService, et met à jour la liste des ingrédients dans la session.
removeFromGrocery	Retire une recette de la liste de courses.	Supprime la recette de la liste, met à jour et renvoie la liste des ingrédients à la session.
resetGroceryList	Réinitialise la liste de courses de l'utilisateur.	Réinitialise la liste de courses dans la base de données, puis met à jour et renvoie la liste dans la session.
printGroceryList	Génère un PDF de la liste de courses.	Crée un fichier PDF intitulé "Ingredients List" avec une table des ingrédients et quantités. Envoie le PDF à l'utilisateur en réponse.

Résumé de la classe

- But : Cette classe sert de contrôleur pour gérer les actions liées aux comptes d'utilisateur, y compris l'enregistrement, la connexion, la gestion de la liste de courses et la génération de rapports PDF. Elle interagit avec le service AccountService pour effectuer les opérations métiers nécessaires.

- Dépendances : La classe dépend du service AccountService, qui encapsule la logique métier liée aux comptes d'utilisateur, et du DAO AccountDAO_InDB pour accéder aux données des comptes.
- Fonctionnalités : Elle permet à l'utilisateur de s'inscrire, de gérer sa liste de courses (ajouter, supprimer des recettes), de réinitialiser sa liste de courses, et de générer un rapport PDF de ses ingrédients. Elle gère aussi la déconnexion et l'affichage du tableau de bord administrateur.

En résumé, la classe AccountServlet est un contrôleur de servlet qui gère les interactions de l'utilisateur liées à son compte, offrant des fonctionnalités telles que l'inscription, la gestion de la liste de courses et la génération de rapports PDF, tout en déléguant les opérations complexes à la couche de service.

GetLocaleRequest

Description

La classe GetLocaleRequest est un contrôleur de servlet dans une application Java EE, chargé de gérer la sélection de la langue de l'utilisateur. Lorsqu'un utilisateur choisit une langue via une interface, cette servlet permet de stocker la langue sélectionnée dans la session de l'utilisateur et le redirige vers la page précédente ou la page d'accueil.

Détail de la classe

- Elle récupère le paramètre userlocale de la requête HTTP, qui représente la langue choisie par l'utilisateur.
- Elle récupère également l'en-tête Referer, qui permet de savoir d'où provient la requête, pour rediriger l'utilisateur vers la même page après la modification de la langue.
- Si le paramètre userlocale est absent (null), la langue par défaut est définie sur "en" (anglais).
- La langue sélectionnée est ensuite stockée dans la session de l'utilisateur sous l'attribut locale.
- Enfin, si un referer est présent, l'utilisateur est redirigé vers la page d'où il vient. Sinon, il est redirigé vers la page d'accueil (index.jsp).

Résumé de la classe

- But : La classe sert à gérer la sélection de la langue de l'utilisateur. Elle permet de stocker la langue choisie dans la session et redirige l'utilisateur vers la page précédente ou la page d'accueil en fonction de l'en-tête Referer.
- Fonctionnalités :
 - Elle permet de récupérer la langue choisie par l'utilisateur depuis un paramètre de requête.
 - Elle permet de stocker cette langue dans la session de l'utilisateur.
 - Elle permet de rediriger l'utilisateur vers la page d'où il vient ou vers la page d'accueil.
- Dépendances : La servlet dépend des requêtes et réponses HTTP, ainsi que de la session de l'utilisateur pour stocker l'information relative à la langue sélectionnée.

En résumé, la classe GetLocaleRequest permet de gérer la sélection de la langue pour un utilisateur, en stockant cette information dans la session et en redirigeant l'utilisateur vers la page appropriée après la modification de la langue.

LoadDashboardPage

Description

La classe LoadDashboardPage est un contrôleur de servlet dans une application Java EE. Elle gère les actions liées à l'administration, comme l'approbation, la suppression, et la création de comptes utilisateurs. Elle permet aussi de charger dynamiquement des pages du tableau de bord de l'administrateur, en fonction de l'action demandée.

Détail des méthodes de la classe

Action	Description	Processus
loadPage	Charge une page spécifique du tableau de bord administrateur.	Si la page demandée est approveAccounts, charge la liste des comptes inactifs. Si la page est deleteAccounts, charge la liste complète des comptes. Redirige avec request.getRequestDispatcher().forward().
approveUserAccount	Approuve un compte utilisateur.	Appelle approveAccount de AccountService pour mettre à jour l'état du compte. Recharge la liste des comptes inactifs et redirige vers la page d'approbation des comptes.
deleteUserAccount	Supprime un compte utilisateur.	Utilise deleteAccount pour supprimer un compte. Met à jour la liste des comptes et redirige vers la page de suppression des comptes.
createUserAccount	Création d'un nouveau compte utilisateur par un administrateur.	Vérifie si le mot de passe et sa confirmation correspondent. Si c'est le cas, appelle submitAccount pour enregistrer le compte. Stocke le résultat en session, met à jour la liste des comptes et redirige vers la page des comptes à ajouter.

Résumé de la classe

- But : La classe sert à gérer diverses actions dans le tableau de bord de l'administrateur. Elle permet de charger des pages, d'approuver des comptes, de supprimer des comptes et de créer de nouveaux comptes.
- Fonctionnalités :
 - Charger dynamiquement des pages dans l'interface d'administration, comme la gestion des comptes utilisateurs.
 - Approuver, supprimer ou créer des comptes utilisateurs via des actions spécifiques.
 - Mettre à jour la session et rediriger l'utilisateur vers les pages appropriées.
- Dépendances : La servlet dépend de AccountService et de AccountDAO_InDB pour interagir avec les données liées aux comptes utilisateurs, ainsi que des requêtes et réponses HTTP pour récupérer et envoyer les informations nécessaires.

En résumé, la classe LoadDashboardPage est responsable de la gestion des actions administratives dans un tableau de bord, permettant l'approbation, la suppression et la création de comptes, ainsi que le redimensionnement dynamique de l'interface en fonction des actions demandées.

LoginServlet

Description

La classe LoginServlet est un contrôleur qui gère le processus de connexion d'un utilisateur à une application Java EE. Elle permet de vérifier les informations d'identification de l'utilisateur (nom d'utilisateur et mot de passe) et de gérer les actions appropriées en fonction des résultats de cette vérification.

Détail des méthodes de la classe

Étapes du processus :

1. Récupère les paramètres username et password envoyés par le formulaire de connexion.
2. Vérifie si les informations d'identification sont valides en appelant la méthode `verifyPassword` du service `AccountService`.
3. Si les informations ne sont pas valides :
 - L'utilisateur est redirigé vers la page de connexion avec un message d'erreur.
4. Si le mot de passe est correct :
 - L'utilisateur est authentifié et un objet `Account` est récupéré à partir du nom d'utilisateur.
 - Si le compte est désactivé (statut 0), l'utilisateur est redirigé vers la page de connexion avec un message d'erreur.
 - Si le compte est actif :
 - La session est mise à jour avec des informations comme l'état de connexion, le compte utilisateur et une liste d'ingrédients (extrait des recettes).
 - Si l'utilisateur est un administrateur (identifiant utilisateur 1), il est redirigé vers la page du tableau de bord administrateur.
 - Si l'utilisateur est un utilisateur normal, il est redirigé vers la page d'accueil (`index.jsp`).

Résumé de la classe

- But : La classe LoginServlet gère la connexion des utilisateurs, vérifie leurs informations d'identification et redirige les utilisateurs vers différentes pages en fonction de leur statut et rôle (utilisateur régulier ou administrateur).
- Fonctionnalités principales :
 - Validation du mot de passe et de l'état du compte.
 - Mise à jour de la session avec des informations pertinentes, telles que le compte de l'utilisateur et une liste d'ingrédients.
 - Redirection vers différentes pages en fonction du rôle de l'utilisateur (administrateur ou utilisateur régulier).
- Dépendances :
 - La servlet dépend de la classe `AccountService` pour la gestion des comptes et des informations de connexion.

- Elle utilise également la méthode `verifyPassword` pour valider les informations d'identification et `fetchAccountByUsername` pour récupérer les détails du compte de l'utilisateur.

RecipeServlet

Description

La classe RecipeServlet est un contrôleur pour gérer les différentes actions liées aux recettes dans une application Java EE. Elle permet d'afficher des recettes, de les filtrer par critères (ingrédient, restriction, catégorie, etc.), d'ouvrir une recette spécifique, et d'interagir avec les étapes de préparation des recettes.

Détail des méthodes de la classe

Action	Description	Process
showRecipes	Récupère la liste de toutes les recettes.	Utilise la méthode fetchRecipeList() du service RecipeService pour récupérer toutes les recettes, puis enregistre la liste dans l'attribut de session recipes et redirige vers recipes-search.jsp.
searchByIngredient	Recherche des recettes contenant un ingrédient spécifique.	Récupère l'ingrédient depuis la requête et effectue une recherche des recettes correspondantes, puis les enregistre dans la session et redirige vers la page de recherche des recettes.
searchByRestriction	Recherche des recettes selon une restriction spécifique (végétarien, sans gluten, etc.).	Effectue une recherche par restriction et enregistre les recettes correspondantes dans la session, redirige ensuite vers la page de recherche des recettes.
searchTopPicks	Recherche des recettes avec la meilleure note (note moyenne de 5).	Utilise fetchRecipesByAverageRating() pour récupérer les recettes avec une note de 5, enregistre les résultats dans la session, puis redirige vers la page de recherche.
searchByCategory	Recherche des recettes par catégorie (dessert, plat principal, etc.).	Récupère la catégorie de recette et effectue la recherche par cette catégorie, enregistre les résultats dans la session, puis redirige vers la page de recherche des recettes.
openRecipe	Ouvre une recette spécifique.	Utilise fetchRecipeById() pour récupérer la recette par son ID, vérifie si l'utilisateur est connecté et s'il a déjà ajouté la recette à sa liste d'épicerie, puis redirige vers recipe.jsp.
lastInstruction	Affiche l'étape précédente de préparation.	Diminue le compteur d'étape de préparation, redirige ensuite vers recipe.jsp pour afficher l'étape mise à jour.
nextInstruction	Affiche l'étape suivante de préparation.	Augmente le compteur d'étape de préparation, puis redirige vers recipe.jsp pour afficher l'étape mise à jour.

Résumé de la classe

- But : La classe RecipeServlet gère les actions liées aux recettes, y compris la recherche de recettes par critères, l'affichage des recettes, et la navigation dans les étapes de préparation des recettes.
- Fonctionnalités principales :
 - Recherche de recettes par ingrédient, restriction, catégorie, ou popularité.
 - Affichage d'une recette spécifique avec la possibilité de suivre les étapes de préparation.
 - Interaction avec la liste d'épicerie de l'utilisateur (ajout de recettes à la liste).

- Navigation dans les étapes de la recette avec un compteur pour les instructions.
- Dépendances :
 - Utilise le service RecipeService pour interagir avec la base de données des recettes.
 - Gère les informations de session de l'utilisateur, telles que l'état de connexion et les recettes ajoutées à la liste d'épicerie.

Base de données

Vue d'ensemble de la structure

La base de données est composée de plusieurs tables conçues pour stocker des informations liées aux comptes, recettes, ingrédients, restrictions, évaluations, instructions et listes de courses. Chaque table est structurée pour contenir des informations spécifiques, et des clés étrangères sont utilisées pour établir des relations entre les tables.

```
CREATE TABLE Account (  
  userID INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(255) NOT NULL,  
  firstname VARCHAR(255) NOT NULL,  
  lastname VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  status INT NOT NULL,  
  securityQuestion VARCHAR(255) NOT NULL,  
  securityAnswer VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE Recipe (  
  recipeID INT PRIMARY KEY AUTO_INCREMENT,  
  title VARCHAR(255) NOT NULL,  
  description TEXT NOT NULL,  
  category VARCHAR(255),  
  difficulty VARCHAR(50),  
  prepTime DOUBLE,  
  servings DOUBLE,  
  status INT NOT NULL  
);
```

```
CREATE TABLE Ingredient (  
  ingredientID INT PRIMARY KEY AUTO_INCREMENT,  
  recipeID INT NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  quantity VARCHAR(50) NOT NULL,  
  unit VARCHAR(50) NOT NULL,  
  FOREIGN KEY (recipeID) REFERENCES Recipe(recipeID) ON DELETE CASCADE  
);
```

```
CREATE TABLE Restriction (  
  restrictionID INT PRIMARY KEY AUTO_INCREMENT,  
  restrictionType VARCHAR(255) NOT NULL,  
  recipeID INT NOT NULL,  
  FOREIGN KEY (recipeID) REFERENCES Recipe(recipeID) ON DELETE CASCADE  
);
```

```
CREATE TABLE Rating (  
  ratingID INT PRIMARY KEY AUTO_INCREMENT,  
  userID INT NOT NULL,  
  recipeID INT NOT NULL,  
  rate INT NOT NULL,  
  FOREIGN KEY (userID) REFERENCES Account(userID) ON DELETE CASCADE,  
  FOREIGN KEY (recipeID) REFERENCES Recipe(recipeID) ON DELETE CASCADE  
);
```

```
CREATE TABLE Instruction (  
  instructionID INT PRIMARY KEY AUTO_INCREMENT,  
  instructions TEXT NOT NULL,  
  recipeID INT NOT NULL,  
  FOREIGN KEY (recipeID) REFERENCES Recipe(recipeID) ON DELETE CASCADE  
);
```

```
CREATE TABLE GroceryList (  
  groceryListID INT PRIMARY KEY AUTO_INCREMENT,  
  recipesList TEXT,  
  userID INT NOT NULL,  
  FOREIGN KEY (userID) REFERENCES Account(userID) ON DELETE CASCADE  
);
```

Model

Voici un résumé bref de chaque classe dans votre package Model :

Account

Attributs

- userID
- firstName
- lastName
- username
- email
- password
- securityQuestion
- securityAnswer
- status (pour l'état du compte)
- groceryList : Liste des objets Recipe

Description

Représente un compte utilisateur, incluant les informations personnelles, les informations de sécurité du compte et une liste de recettes dans la liste de courses de l'utilisateur. Plusieurs constructeurs sont fournis pour différents cas d'utilisation. Des méthodes sont incluses pour obtenir et définir ces attributs.

Ingredient

Attributs :

- ingredientID
- name
- quantity
- unit

Description : Représente un ingrédient utilisé dans une recette, avec des détails sur son nom, sa quantité et son unité de mesure. Contient des constructeurs et des méthodes pour gérer les données des ingrédients.

Rating

Attributs

- ratingId
- userId
- rate

Description

Représente une évaluation donnée à une recette, incluant l'ID de l'utilisateur et la valeur de l'évaluation (un entier). Fournit des méthodes pour obtenir et définir les informations d'évaluation.

Recipe

Attributs

- recipeID
- title
- description
- category
- difficulty
- prepTime
- servings
- status.
- Listes
- ingredientList
- restrictionList
- ratingList
- instruction

Description

Représente une recette, incluant son titre, sa description, les ingrédients, les restrictions, les évaluations, les instructions et d'autres métadonnées. Fournit des méthodes pour ajouter/retirer des ingrédients, des restrictions, des évaluations et des instructions. Inclut également une méthode pour calculer la note moyenne de la recette.

Restriction

Attributs

- restrictionID
- restrictionType

Description

Représente une restriction (par exemple, des restrictions alimentaires) liée à une recette. Contient des méthodes pour gérer les données de restriction, y compris son ID et son type.

Chaque classe est conçue pour modéliser des entités dans une application de recettes, où Account contient les données de l'utilisateur, Recipe représente les recettes, Ingredient contient les détails des ingrédients, Rating capture les évaluations des utilisateurs, et Restriction définit les limitations des recettes. Les méthodes dans ces classes supportent les opérations de base nécessaires pour créer, mettre à jour et récupérer les données dans l'application.

Scripts

Scripts de adminPanel.jsp

Le code JavaScript dans le fichier .jsp fourni se compose de quatre scripts principaux, chacun ayant une fonction différente pour gérer le panneau d'administration :

1. Script de vue dynamique du tableau de bord :
 - Ce script écoute les événements de clic sur les sections de la barre latérale (liens avec la classe .panel-section).
 - Lorsqu'une section est cliquée, le script empêche l'action par défaut, récupère l'URL associée à partir de l'attribut data-url, et charge le contenu dans l'élément .dashboard-view en utilisant l'API fetch.
 - Un "skeleton loader" est affiché pendant que le nouveau contenu est chargé. Si le contenu est récupéré avec succès, il remplace le contenu actuel dans la vue du tableau de bord. En cas d'erreur, un message d'erreur est affiché.
2. Script d'approbation de compte :
 - Cette fonction est déclenchée pour approuver un compte utilisateur en envoyant une requête POST au serveur avec le userId et une valeur dashboardAction (ici approveUserAccount).
 - En cas de succès, le script récupère et met à jour le contenu de la page "approbation des comptes" dans la vue du tableau de bord. Si une erreur se produit, une alerte avec le message d'erreur est affichée.
3. Script de suppression de compte :
 - Cette fonction supprime un compte utilisateur en envoyant une requête POST au serveur avec le userId et une valeur dashboardAction (ici deleteUserAccount).
 - De manière similaire au script d'approbation, après la suppression réussie du compte, il récupère et met à jour la page "suppression des comptes". En cas d'erreur, une alerte avec le message d'erreur est affichée.
4. Script de création de compte :
 - Cette fonction gère la création d'un nouveau compte utilisateur. Elle collecte les données du formulaire à partir d'un formulaire ayant l'ID createAccountForm et ajoute une valeur dashboardAction (ici createUserAccount).
 - Les données du formulaire sont converties en une chaîne URL-encodée et envoyées via une requête POST au serveur.
 - Après la création réussie d'un compte, la partie pertinente de la page (section "ajouter des comptes") est rechargée. En cas d'erreur, une alerte avec le message d'erreur est affichée.

Ces scripts fonctionnent ensemble pour fournir une fonctionnalité dynamique et interactive pour le panneau d'administration, permettant la gestion des comptes (approbation, suppression et création de comptes) et des mises à jour de contenu sans rechargement complet de la page.

Scripts de recipes-search.jsp

Le code JavaScript dans ce fichier .jsp contient deux fonctions principales, utilisées pour gérer l'interactivité avec les filtres de catégorie dans la section des recettes :

1. Fonction toggleSubCategory :

- Cette fonction permet de basculer l'affichage des sous-catégories (comme "Régime", "Viande", "Légumes", etc.) lorsque l'utilisateur clique sur le titre de la sous-catégorie.
- Elle permet d'ouvrir ou de fermer les sous-catégories en ajoutant ou retirant la classe open de l'élément.
- Lorsque la sous-catégorie est ouverte, le symbole plus (+) devient un symbole moins (-) pour indiquer qu'il est possible de la fermer.

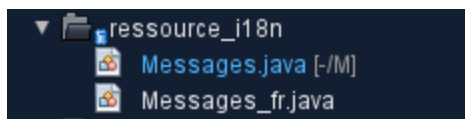
2. Fonction submitCategoryForm :

- Cette fonction est utilisée pour soumettre le formulaire de recherche de recettes avec les filtres de catégorie sélectionnés (restriction alimentaire, ingrédient, ou type de recette).
- Les valeurs de ces filtres sont définies dans les champs cachés du formulaire, et chaque filtre correspond à une action (searchByRestriction, searchByIngredient, ou searchByCategory).
- Une fois que les valeurs sont définies, le formulaire est soumis via `document.getElementById('categoryForm').submit()` pour déclencher une recherche avec les paramètres de filtre spécifiés.

Ces scripts sont utilisés pour créer une interface dynamique permettant de filtrer les recettes en fonction de différentes catégories telles que le régime alimentaire, les ingrédients, ou le type de recette.

Internationalisation

L'internationalisation (i18n) consiste à concevoir une application ou un site web afin qu'il puisse être facilement adapté à différentes langues et régions sans nécessiter de modifications majeures du code. Cela implique l'utilisation de *resource bundles* pour stocker les traductions dans des fichiers distincts pour chaque langue (par exemple, Messages pour l'anglais, Messages_fr pour le français). La locale de l'utilisateur est stockée dans la session, permettant ainsi de charger dynamiquement les traductions appropriées en fonction de la langue sélectionnée. La gestion de la langue se fait via un paramètre de requête (par exemple, userlocale), et après la sélection de la langue, l'application redirige l'utilisateur vers la page qu'il consultait, affichant ainsi le contenu dans la nouvelle langue. L'utilisation des balises JSP comme `<fmt:setLocale>` et `<fmt:setBundle>` permet de charger les messages traduits de manière transparente, offrant ainsi une expérience utilisateur cohérente et multilingue.



```
<fmt:setLocale value = "${sessionScope.locale}"/>
<fmt:setBundle basename = "ressource_i18n.Messages"/>
```

Conclusion

En conclusion, notre application garantit une expérience utilisateur fluide et sécurisée, tout en offrant une gestion efficace des recettes et des listes d'épicerie. L'architecture MVC assure une séparation claire des responsabilités, rendant l'application facilement maintenable et évolutive. Avec des fonctionnalités adaptées aux utilisateurs enregistrés et non enregistrés, ainsi qu'un accès sécurisé pour les administrateurs, l'application allie performance, flexibilité et sécurité.